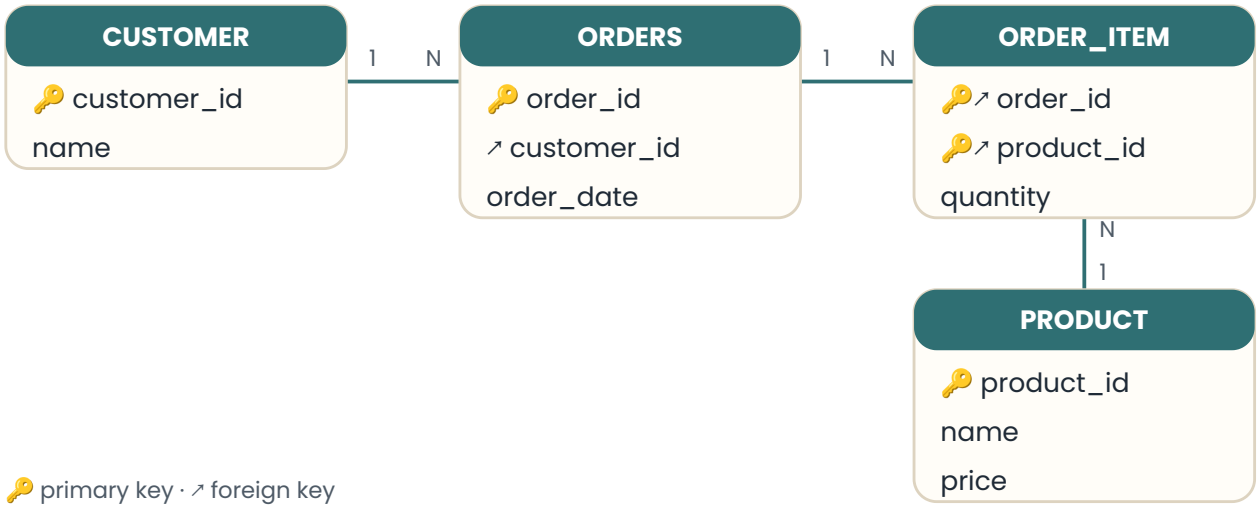
1 Database management systems

Week 1 · Cheat sheet



primary key · ↗ foreign key

Read data

`SELECT name, price FROM product;`	Choose columns from a table
`… WHERE price > 20000`	Keep only rows that match
`… WHERE price >= 8000 AND price <= 20000`	Combine conditions
`… ORDER BY price DESC`	Sort, highest first
`SELECT COUNT(*) FROM orders;`	Count rows

Combine tables

`FROM customer c JOIN orders o ON c.customer_id = o.customer_id`	Rows whose keys match
`CREATE VIEW cheap AS SELECT … ;`	Save a query with a name

Change data

`INSERT INTO product (product_id, name, price) VALUES (60, 'Pencil', 2000);`	Add a row
`UPDATE product SET price = 9000 WHERE product_id = 10;`	Change rows (always with WHERE)
`DELETE FROM product WHERE product_id = 60;`	Remove rows (always with WHERE)

Define structure and rules

`product_id INTEGER PRIMARY KEY`	Unique, non-null identifier
`customer_id INTEGER REFERENCES customer(customer_id)`	Foreign key to another table
`name VARCHAR(60) NOT NULL`	A value is required
`price INTEGER CHECK (price >= 0)`	Validate a condition

Design: ER → tables

`Entity → table`	Attributes become columns, key becomes PK
`1:N → FK on the N side`	order.customer_id REFERENCES customer
`N:M → junction table`	Both keys as FKs, often a composite PK
`Multivalued → new table`	One row per value (patient_phone)
`Weak entity → PK = owner + discriminator`	room(ward_id, room_no)
`Derived → not stored`	Calculate it in queries (age)

Transactions

`BEGIN; … COMMIT;`	Apply all steps together
`BEGIN; … ROLLBACK;`	Undo everything since BEGIN

DDL: build the structure

`CREATE TABLE t (id INTEGER PRIMARY KEY, …);`	Create a table
`ALTER TABLE t ADD COLUMN phone VARCHAR(20);`	Add a column
`TRUNCATE TABLE t;`	Empty it, keep the table
`DROP TABLE t;`	Remove the table and its data
`… ON DELETE CASCADE | RESTRICT | SET NULL`	What happens to children when a parent goes

When something goes wrong

Symptom	Likely cause	Fix
FOREIGN KEY constraint failed	The referenced row does not exist	Insert the parent row first (for example, the customer)
CHECK constraint failed	A value breaks a rule, such as a negative price	Fix the value or the rule
Every row changed	UPDATE or DELETE without WHERE	Test the condition with SELECT first; work inside a transaction